



Gerenciamento de dependências e builds

Programação Web II

Gerenciamento de dependências e builds

Programação Web II

O que são dependências?

Dependências são **bibliotecas externas** reutilizáveis criadas por outros desenvolvedores para acelerar o desenvolvimento.

Exemplo:

Biblioteca	Função
Jackson	JSON
Lombok	reduzir código
JUnit	testes
Spring Boot	APIs
Hibernate	banco de dados

Gerenciamento de dependências e builds

Programação Web II

Sem

dependências

Precisaríamos implementar tudo manualmente.

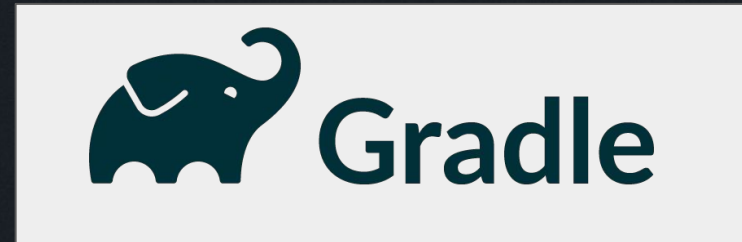
Analogia: Dependências são como *apps* que adicionamos ao nosso projeto Java.

Gerenciamento de dependências e builds

Programação Web II

Gerenciamento de Dependências com Maven/Gradle

Maven/Gradle é uma ferramenta de gerenciamento de projetos e automação de builds. Ela facilita a inclusão de bibliotecas externas e o controle de versões.



Gerenciamento de dependências e builds

Programação Web II

O que o Gradle faz?

O Gradle automatiza tarefas do projeto Java.

Principais responsabilidades

- baixar bibliotecas automaticamente
- gerenciar versões
- compilar projeto
- executar testes
- gerar builds da aplicação

Exemplo:

```
implementation 'com.fasterxml.jackson.core:jackson-databind:2.18.2'
```

Gradle:

- baixa *Jackson*
- adiciona ao projeto
- resolve dependências automaticamente

Analogia: Gradle funciona como um “gerenciador de pacotes”.

Gerenciamento de dependências e builds

Programação Web II

Fluxo de uma aplicação backend

```
Usuário
↓
Console / Postman
↓
Main
↓
Service
↓
Http Client
↓
API Externa
↓
JSON
↓
Jackson
↓
Objeto Java
↓
Console / Resposta
```

Gerenciamento de dependências e builds

Programação Web II

Responsabilidade das classes

Organização da aplicação:

Classe	Responsabilidade
Main	Integração/Menu
ProductService	regras do produto
TodoService	regras de tarefas
HttpClient	comunicação HTTP
DTO/Model	representar troca de dados
ObjectMapper	converter JSON
Response	tipo específico de DTO

Gerenciamento de dependências e builds

Programação Web II

DTO x Response x Model

→ *DTO (Data Transfer Object)*

DTO é um objeto utilizado para transportar dados entre sistemas, APIs ou camadas da aplicação.

Seu objetivo é:

- enviar dados
- receber dados
- representar JSONs
- facilitar integração entre sistemas

Gerenciamento de dependências e builds

Programação Web II

DTO x Response x Model

→ DTO (*Data Transfer Object*)

Exemplo de DTO:

- API retorna:

```
{  
  "temp_C": "22",  
  "humidity": "80"  
}
```

- DTO:

```
public class CurrentCondition {  
  
    private String temp_C;  
    private String humidity;  
  
}
```

Gerenciamento de dependências e builds

Programação Web II

DTO x Response x Model

→ Response

Response é um tipo específico de **DTO**.

Ele normalmente representa:

- resposta de API
- retorno HTTP
- estrutura completa do JSON

Exemplo de Response:

```
public class WeatherResponse {  
    private List<CurrentCondition> currentCondition;  
}
```

```
{  
  "currentCondition": [  
    {  
      "temp_C": "22",  
      "humidity": "80"  
    },  
    {  
      "temp_C": "22",  
      "humidity": "80"  
    },  
    {  
      "temp_C": "22",  
      "humidity": "80"  
    },  
    {  
      "temp_C": "22",  
      "humidity": "80"  
    }  
  ]  
}
```

Gerenciamento de dependências e builds

Programação Web II

DTO x Response

Exemplo de Response:

Json:

```
{
  "products": [
    {
      "id": 101,
      "title": "Apple AirPods Max Silver",
      "category": "mobile-accessories"
      ...
    },
    {...},
    {...},
    {...}
  ]
  // 23 results
},
"total": 23,
"skip": 0,
"limit": 23
}
```

Response:

```
public class ProductResponse {

    private List<ProductDto> products;
    private Int total;
    private Int skip;
    private Int limit;
}
```

Dto:

```
public class ProductDto {

    private Int id;
    private String title;
    private String category;
}
```

Gerenciamento de dependências e builds

Programação Web II

DTO x Response x Model

→ Model

Model representa o domínio da aplicação.

Ele NÃO precisa seguir exatamente o JSON da API.

Representa:

- regra de negócio
- dados internos da aplicação

Exemplo de Model:

```
public class Weather {  
  
    private String cidade;  
    private String temperatura;  
    private String descricao;  
}
```


Gerenciamento de dependências e builds

Programação Web II

DTO x Response x Model

→ Fluxo real de backend

```
API Externa  
↓  
DTO / Response  
↓  
Service  
↓  
Model  
↓  
Aplicação
```

Gerenciamento de dependências e builds

Programação Web II

DTO x Response x Model

→ Diferença principal

Classe	Responsabilidade
DTO	transportar dados
Response	representar resposta da API
Model	representar domínio da aplicação

→ Analogia

- **DTO** → caixa de transporte de dados
- **Response** → resposta recebida da API
- **Model** → objeto interno da aplicação

Todo **Response** é um **DTO**.
Mas nem todo **DTO** é um **Response**.

Serialização x Desserialização

Conversões JSON:

Conceito	Conversão
Serialização	Objeto → JSON
Desserialização	JSON → Objeto

Exemplos:

→ Serialização

```
mapper.writeValueAsString (produto)
```

→ Desserialização

```
mapper.readValue (json,  
ProductResponse.class)
```

Gerenciamento de dependências e builds

Programação Web II

Sem Jackson x Com Jackson

→ Sem Jackson

```
String json = ""  
{  
  "nome":"Notebook"  
}  
"";
```

→ Com Jackson

```
String json = mapper.writeValueAsString (produto);
```

Vantagens do Jackson

- automático
- menos erros
- código limpo
- mais profissional

Gerenciamento de dependências e builds

Programação Web II

Manipulação de dados com Jackson

Jackson é uma biblioteca popular para converter objetos Java em JSON e vice-versa.

A classe *ObjectMapper* é a responsável por fazer essa conversão de maneira simples, como no exemplo a seguir.

```
import com.fasterxml.jackson.databind.ObjectMapper;

public class ExemploJackson {
    public static void main(String[] args) {
        try {
            ObjectMapper mapper = new ObjectMapper();
            Produto produto = new Produto("Notebook", 2500.0);
            String json = mapper.writeValueAsString(produto);
            System.out.println(json);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Links para consulta: [baeldung](#), [jackson-annotations](#), [jackson](#)

Manipulação de dados com Jackson

Programação Web II

Conversão de JSON para objeto:

```
String json = "{\"nome\":\"Notebook\",\"preco\":2500.0}";  
Produto produto = mapper.readValue(json, Produto.class);
```

Anotações Jackson para manipulação de JSON

Jackson fornece diversas anotações para personalizar a maneira como os campos e classes são serializados e desserializados. Aqui estão algumas das principais:

@JsonProperty

define o nome que será usado no JSON para um campo específico. É útil quando o nome do campo na classe Java difere daquele usado no JSON.

```
import com.fasterxml.jackson.annotation.JsonProperty;  
  
public class Produto {  
  
    @JsonProperty("nome_produto")  
    private String nome;  
  
    @JsonProperty("preco_produto")  
    private double preco;  
  
    // Construtor, getters e setters  
}
```

Anotações Jackson para manipulação de JSON

Programação Web II

@JsonIgnore

Ignora um campo durante a serialização e desserialização.

Isso é útil para excluir informações sensíveis ou desnecessárias no JSON.

```
import com.fasterxml.jackson.annotation.JsonIgnore;

public class Produto {

    private String nome;
    private double preco;

    @JsonIgnore
    private String codigoInterno;

    // Construtor, getters e setters
}
```


Anotações Jackson para manipulação de JSON

Programação Web II

@JsonFormat

Usado para especificar o formato de datas ou outros tipos de valores durante a serialização e desserialização.

Por exemplo, formatar uma data para o padrão dd/MM/yyyy.

```
import com.fasterxml.jackson.annotation.JsonFormat;
import java.util.Date;

public class Produto {

    private String nome;
    private double preco;

    @JsonFormat(shape = JsonFormat.Shape.STRING, pattern = "dd/MM/yyyy")
    private Date dataLancamento;

    // Construtor, getters e setters
}
```


Redução de Código com Lombok

Programação Web II

Lombok é uma biblioteca que reduz a necessidade de escrever métodos repetitivos como *getters*, *setters* e *toString*.

Configuração do Lombok:

- Maven **pom.xml**:

```
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <version>1.18.20</version>
  <scope>provided</scope>
</dependency>
```

- Gradle **build.gradle**:

```
dependencies {
  compileOnly("org.projectlombok:lombok:1.18.42" )
  annotationProcessor ("org.projectlombok:lombok:1.18.42" )
}
```

Redução de Código com Lombok

Programação Web II

Exemplo com Lombok

```
import lombok.Data;

@Data
public class Produto {
    private String nome;
    private double preco;

    public Produto(String nome, double preco) {
        this.nome = nome;
        this.preco = preco;
    }
}
```

Consultar [documentação](#) para mais detalhes sobre anotação `@Data` e a [documentação](#) completa.

Gerenciamento de dependências e builds

Programação Web II

Sem Lombok x Com Lombok

→ Sem Lombok

```
getters  
setters  
toString  
constructors  
equals/hashCode
```

→ Com Lombok

```
@Data  
@NoArgsConstructor  
@AllArgsConstructor
```

Benefícios

- menos código repetitivo
- mais produtividade
- classes mais limpas

Gerenciamento de dependências e builds

Programação Web II

Estrutura Backend Java

```
src/main/java
├── Main.java
├── service/
│   ├── ProductService.java
│   └── TodoService.java
├── model/
│   ├── Product.java
│   └── Todo.java
├── dto/
│   ├── ProductResponse.java
│   └── TodoResponse.java
└── http/
    └── HttpClientUtil.java
```


Exercícios

Exercícios

Gerenciamento de dependências e builds

Exercício 1: Sistema de Usuários + Posts (JSONPlaceholder)

Contexto: Você foi contratado para desenvolver um sistema que consulta:

- usuários
- posts realizados pelos usuários

A aplicação deverá consumir a API pública: [JsonPlaceholder](https://jsonplaceholder.typicode.com/)

Endpoints utilizados

- **Usuários:** GET /users
- **Posts de um usuário:** GET /posts?userId={id}

Objetivo Geral

Construir uma aplicação Java que:

- consuma APIs REST
- converta JSON em objetos Java
- organize código em camadas
- relacione usuários com posts
- utilize **Jackson** e **Lombok**
- possua menu interativo

Exercícios

Gerenciamento de dependências e builds

Exercício 1: Sistema de Usuários + Posts (JSONPlaceholder)

Estrutura esperada

```
src/  
├── Main.java  
├── service/  
│   ├── UserService.java  
│   └── PostService.java  
├── model/  
│   ├── User.java  
│   └── Post.java  
├── dto/  
│   ├── UserDto.java  
│   └── PostDto.java  
└── http/  
    └── HttpClientUtil.java
```

Exercícios

Gerenciamento de dependências e builds

Exercício 1: Sistema de Usuários + Posts (JSONPlaceholder)

Funcionalidades obrigatórias

→ Menu interativo

```
=====
1 - Listar usuários
2 - Buscar usuário por ID
3 - Listar posts de usuário
4 - Sair
=====
```


Exercícios

Gerenciamento de dependências e builds

Exercício 1: Sistema de Usuários + Posts (JSONPlaceholder)

Funcionalidades obrigatórias

→ Modelagem com Lombok

Criar Classe: *User*

Atributos: id, name, email, phone, website

Criar Classe: *Post*

Atributos: userId, id, title, body

Exercícios

Gerenciamento de dependências e builds

Exercício 1: Sistema de Usuários + Posts (JSONPlaceholder)

Funcionalidades obrigatórias

→ Consumir APIs REST

Utilizar:

- `HttpURLConnection`
- `Scanner`
- `ObjectMapper`

Exercícios

Gerenciamento de dependências e builds

Exercício 1: Sistema de Usuários + Posts (JSONPlaceholder)

Funcionalidades obrigatórias

→ Conversão JSON → Objetos

Usar para desserialização: *ObjectMapper*

Exercícios

Gerenciamento de dependências e builds

Exercício 1: Sistema de Usuários + Posts (JSONPlaceholder)

Funcionalidades obrigatórias

→ Exibir informações formatadas

Usuários:

```
ID: 1  
Nome: Leanne Graham  
Email: leanne@gmail.com
```

Posts:

```
POST ID: 1  
Título: Meu primeiro post  
Descrição: ...
```


Exercício 1: Sistema de Usuários + Posts (JSONPlaceholder)

Funcionalidades obrigatórias

→ Tratamento de erros

Seu sistema deve tratar:

- usuário inexistente
- erro HTTP
- API fora do ar
- entrada inválida
- números negativos

Exercícios

Gerenciamento de dependências e builds

Exercício 1: Sistema de Usuários + Posts (JSONPlaceholder)

★ Desafio Extra

Mostrar:

```
Usuário: Stephannie  
Quantidade de posts: 10
```

Exercícios

Gerenciamento de dependências e builds

Exercício 2: Sistema de Clima + Histórico de Consultas

Contexto: Você deverá criar um sistema de consulta climática utilizando APIs REST.

A aplicação permitirá:

- consultar clima por cidade
- salvar histórico de consultas
- organizar dados em objetos Java
- tratar erros HTTP

API utilizada: <https://wttr.in/>

Documentação: <https://github.com/chubin/wttr.in>

Exemplo: <https://wttr.in/SaoPaulo?format=j1>

Exercício 2: Sistema de Clima + Histórico de Consultas

Objetivo Geral

Criar uma aplicação backend console que:

- consuma API externa
- trabalhe com JSON complexo
- utilize DTOs
- utilize listas
- utilize *Lombok*
- organize código em camadas

Exercícios

Gerenciamento de dependências e builds

Exercício 2: Sistema de Clima + Histórico de Consultas

Estrutura esperada

```
src/main/java/  
├── br/  
│   ├── com/  
│   │   ├── ada/  
│   │   │   ├── Main.java  
│   │   │   ├── service/  
│   │   │   │   └── WeatherService.java  
│   │   │   ├── model/  
│   │   │   │   ├── Weather.java  
│   │   │   │   └── WeatherHistory.java  
│   │   │   ├── dto/  
│   │   │   │   ├── WeatherResponse.java  
│   │   │   │   ├── CurrentCondition.java  
│   │   │   │   ├── WeatherDescription.java  
│   │   │   │   ├── NearestArea.java  
│   │   │   │   └── AreaName.java  
│   │   │   └── http/  
│   │   │       └── HttpClientUtil.java
```

Exercícios

Gerenciamento de dependências e builds

Exercício 2: Sistema de Clima + Histórico de Consultas

Funcionalidades obrigatórias

→ Menu interativo

```
=====
1 - Consultar clima
2 - Ver histórico
3 - Limpar histórico
4 - Sair
=====
```

Exercícios

Gerenciamento de dependências e builds

Exercício 2: Sistema de Clima + Histórico de Consultas

Funcionalidades obrigatórias

→ Consulta dinâmica

Usuário poderá digitar:

```
São Paulo  
Rio de Janeiro  
Curitiba
```

Exercícios

Gerenciamento de dependências e builds

Exercício 2: Sistema de Clima + Histórico de Consultas

Funcionalidades obrigatórias

→ Exibir dados formatados

Mostrar:

```
Cidade: São Paulo  
Temperatura: 22°C  
Descrição: Parcialmente nublado  
Umidade: 80%
```


Exercícios

Gerenciamento de dependências e builds

Exercício 2: Sistema de Clima + Histórico de Consultas

Funcionalidades obrigatórias

→ Histórico de consultas

Salvar cidades consultadas em:

```
List<WeatherHistory>
```

Exercícios

Gerenciamento de dependências e builds

Exercício 2: Sistema de Clima + Histórico de Consultas

Funcionalidades obrigatórias

→ Modelagem com Lombok

Criar Classe: *Weather*

Atributos: *cidade, temperatura, descricao, umidade*

Exercícios

Gerenciamento de dependências e builds

Exercício 2: Sistema de Clima + Histórico de Consultas

Funcionalidades obrigatórias

→ Organização em camadas

Separar:

- Main
- Service
- DTO
- HttpClient
- Model

Exercícios

Gerenciamento de dependências e builds

Exercício 2: Sistema de Clima + Histórico de Consultas

Funcionalidades obrigatórias

→ Tratamento de erros

Tratar:

- cidade inválida
- API indisponível
- erro de conexão
- entrada vazia

Exercícios

Gerenciamento de dependências e builds

Exercício 2: Sistema de Clima + Histórico de Consultas

★ Desafio Extra 1

Mostrar quantidade de consultas realizadas:

```
Total de consultas: 15
```

Exercícios

Gerenciamento de dependências e builds

Exercício 2: Sistema de Clima + Histórico de Consultas

★ Desafio Extra 2

Mostrar cidade mais pesquisada:

Cidade mais pesquisada: São Paulo

Exercícios

Gerenciamento de dependências e builds

Exercício 2: Sistema de Clima + Histórico de Consultas

★ Desafio Extra 3

Permitir múltiplas consultas sem reiniciar sistema.

Obrigada